

Waveshare 2.9" e-Paper (B)

Project Reference — Tri-Color (Red / Black / White) SPI Display

1. Overview

The Waveshare 2.9" e-Paper Module (B) is a tri-color (red, black, white) electronic paper display with 296×128 resolution. It uses microcapsule electrophoretic technology: charged pigment particles suspended in a clear fluid move to the surface of each microcapsule when an electric field is applied, reflecting ambient light just like printed paper. No backlight is required, and the image persists indefinitely with zero power once drawn.

The onboard driver PCB adds a voltage translator so the module works with both 3.3 V and 5 V logic MCUs (Raspberry Pi, Arduino, STM32, ESP32, RP2040, and more). Communication is over 4-wire SPI with additional control lines for reset, data/command, and busy status.

Key capabilities:

- 296 × 128 resolution, tri-color (red / black / white)
- Wide viewing angle — nearly 180°
- Ultra-low power — current only drawn during refresh, zero when idle
- Retains last image indefinitely without power
- 4-wire SPI interface (CPOL = 0, CPHA = 0, SPI mode 0)
- Onboard voltage translator — accepts 3.3 V or 5 V logic
- Full refresh only (no partial refresh on the tri-color variant)
- Ideal for price tags, shelf labels, name badges, dashboards, IoT displays

2. Specifications

Parameter	Value
Operating voltage	3.3 V / 5 V (onboard voltage translator)
Panel input voltage	2.3 V to 3.6 V (rated 3.3 V)
Interface	4-wire SPI (SPI mode 0: CPOL=0, CPHA=0)
SPI clock (max)	~10 MHz (typ. 2-4 MHz recommended)
Display color	Red / Black / White (tri-color)
Resolution	296 × 128 pixels
Display size	2.9 inches (dot pitch 0.227 × 0.226 mm)
Viewing angle	> 170°
Full refresh time	~15 seconds (tri-color full refresh)
Refresh interval	≥ 180 s recommended; at least once per 24 h
Operating environment	0 °C to 40 °C, 35 % to 65 % RH
Storage environment	below 30 °C, below 55 % RH, max 6 months
Outline dimensions (module)	89.0 × 38.0 mm
Display area	66.89 × 29.05 mm

Warning: Never leave the e-Paper powered in a static (non-refreshed) state for long periods — the panel is held at high voltage during refresh and may be damaged if parked there. After drawing, always put the display into sleep mode or remove power. Refresh at least once every 24 hours to prevent ghosting and afterimages.

3. Module Pinout (8-pin header)

The module ships with an 8-wire ribbon cable that terminates in a keyed 8-pin female header. Each pin is labeled on the PCB. Pin colors in the cable are a common convention but may vary between revisions — always verify against the silk-screen labels on your module.

Pin	Cable color	Function
VCC	Grey	3.3 V or 5 V supply input
GND	Brown	Ground
DIN	Blue	SPI MOSI — serial data in to the display
CLK	Yellow	SPI SCK — serial clock
CS	Orange	SPI chip select (active LOW)
DC	Green	Data/command select (HIGH = data, LOW = command)
RST	White	Hardware reset (active LOW)
BUSY	Purple	Busy status output from display (HIGH = busy)

BUSY pin direction: some Waveshare variants drive BUSY HIGH when busy, others LOW. The 2.9" (B) module's default driver code treats HIGH as busy. If your display never finishes refreshing, check the polarity in your driver library and invert if needed.

4. Wiring Diagrams

The display uses standard 4-wire SPI plus three control lines (RST, DC, BUSY). BUSY and DC can be any GPIO pins — the numbers below are the defaults used by the official Waveshare demos.

4.1 Raspberry Pi (40-pin GPIO, BCM numbering)

Display pin	Pi BCM GPIO	Physical pin
VCC	3.3 V	Pin 1 or 17
GND	GND	Pin 6, 9, 14, 20, 25, ...
DIN (MOSI)	GPIO 10 (MOSI)	Pin 19
CLK (SCK)	GPIO 11 (SCLK)	Pin 23
CS	GPIO 8 (CE0)	Pin 24
DC	GPIO 25	Pin 22
RST	GPIO 17	Pin 11
BUSY	GPIO 24	Pin 18

4.2 Arduino UNO / Nano (hardware SPI)

Display pin	Arduino UNO
VCC	5V (or 3.3V)
GND	GND
DIN (MOSI)	D11 (MOSI)
CLK (SCK)	D13 (SCK)
CS	D10
DC	D9
RST	D8
BUSY	D7

4.3 ESP32 (default VSPI pins)

Display pin	ESP32 GPIO
VCC	3V3
GND	GND
DIN (MOSI)	GPIO 23
CLK (SCK)	GPIO 18
CS	GPIO 5
DC	GPIO 17
RST	GPIO 16
BUSY	GPIO 4

5. Refresh Behavior & Image Model

The 2.9" e-Paper (B) only supports **full refresh** — the screen flickers several times during each update to clear any ghost images and drive the red/black/white particles to their final positions. A full refresh typically takes around 15 seconds. Partial refresh is **not** supported on the tri-color variant.

Framebuffer layout

Because the panel has three colors but each memory bit is binary, the image is transmitted as **two separate 1-bit buffers** — one for the black/white channel and one for the red/white channel. Each byte in either buffer controls 8 horizontal pixels.

- **B/W buffer** — bit = 0 → black, bit = 1 → white
- **Red buffer** — bit = 0 → red, bit = 1 → no red (white/black shows through)
- **Buffer size** — $296 \times 128 / 8 = 4,736$ bytes per channel, 9,472 bytes total

If a pixel is black in the B/W buffer *and* red in the red buffer, the red buffer generally wins. Library wrappers like `waveshare_epd` handle the split automatically when you pass in a PIL image with red pixels.

Long-term care

- Refresh at least once every 24 hours to avoid ghosting.
- Before long-term storage, clear the display to white so no pixels are parked in a non-neutral state.
- Always call `sleep()` after your last refresh — do not leave the panel powered and idle.
- Avoid direct sunlight and temperatures outside 0–40 °C; UV light can damage the front film.

6. SPI Timing & Control Pins

The display is a write-only SPI peripheral — the host sends commands and pixel data; no read-back is used. SPI mode is 0 (CPOL = 0, CPHA = 0), MSB first.

Signal	Direction	Role
CS	MCU → EPD	Active-LOW chip select. Pull LOW before each command/data byte.
DC	MCU → EPD	LOW = the next SPI byte is a command; HIGH = data/parameter.
RST	MCU → EPD	Active-LOW hardware reset. Pulse LOW briefly (~10 ms) at startup.
BUSY	EPD → MCU	Driven HIGH by the display during refresh. Poll until LOW before the next command.
CLK	MCU → EPD	SPI clock.
DIN	MCU → EPD	SPI MOSI — command byte when DC is LOW, data byte when DC is HIGH.

Important: always wait for BUSY to go LOW before sending the next command. Sending data mid-refresh can corrupt the image or interrupt the waveform, potentially damaging the panel. If your reset pulse is too long, the onboard power-control circuit will cut power instead of resetting. Keep RST LOW for only about 10 ms.

7. Example Code

7.1 Raspberry Pi — Python (waveshare_epd library)

Enable SPI with `sudo raspi-config`, then clone the Waveshare driver repo:

```
sudo apt install python3-pip python3-pil python3-numpy
sudo pip3 install spidev gpiozero

git clone https://github.com/waveshare/e-Paper.git
cd e-Paper/RaspberryPi_JetsonNano/python/examples

from waveshare_epd import epd2in9b_V3
from PIL import Image, ImageDraw, ImageFont
import time

epd = epd2in9b_V3.EPD()
epd.init()
epd.Clear()

# Width/height are swapped because the panel is used in landscape
W, H = epd.height, epd.width # 296 x 128

# Two separate 1-bit images: black layer and red layer
blackimage = Image.new('1', (W, H), 255) # 255 = white background
redimage    = Image.new('1', (W, H), 255)

font = ImageFont.truetype(
    "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf", 24)

ImageDraw.Draw(blackimage).text((10, 10), "Hello, e-Paper!", font=font, fill=0)
ImageDraw.Draw(redimage).text((10, 50), "Red text here", font=font, fill=0)
ImageDraw.Draw(blackimage).rectangle((5, 5, W - 5, H - 5), outline=0)

epd.display(epd.getbuffer(blackimage), epd.getbuffer(redimage))
time.sleep(2)

epd.sleep() # always sleep the panel when done
```

7.2 Arduino (official Waveshare epd2in9b_V3 demo)

Download the demo archive from the [Waveshare wiki](#) and open `Arduino/epd2in9b_V3/epd2in9b_V3.ino`. A stripped-down version:

```
#include <SPI.h>
#include "epd2in9b_V3.h"    // from Waveshare demo
#include "imagedata.h"      // IMAGE_BLACK[] and IMAGE_RED[] arrays

Epd epd;

void setup() {
    Serial.begin(115200);

    if (epd.Init() != 0) {
        Serial.println("e-Paper init failed");
        while (1);
    }

    epd.ClearFrame();
    epd.DisplayFrame(IMAGE_BLACK, IMAGE_RED);
    delay(2000);

    epd.Sleep();    // put panel to sleep; do not leave it powered idle
}

void loop() {
    // e-Paper holds the image with no power; nothing to do here.
}
```

7.3 ESPHome (for Home Assistant users)

```
spi:
  clk_pin: GPIO18
  mosi_pin: GPIO23

display:
  - platform: waveshare_epaper
    cs_pin: GPIO5
    dc_pin: GPIO17
    busy_pin: GPIO4
    reset_pin: GPIO16
    model: 2.90inbc          # tri-color 2.9" (B)
    update_interval: 300s    # refresh every 5 minutes
    lambda: |-
      it.print(0, 0, id(font1), "Hello from ESPHome!");
```

8. Troubleshooting

Display stays blank after flashing code: Check the BUSY pin wiring and direction — it must be an INPUT on the MCU. If the library sits forever in a busy-wait loop, BUSY polarity may be inverted on your revision.

Reset never completes (power LED dims / cycles): Your RST pulse is too long. The onboard power-switch circuit cuts power when RST is held LOW too long. Shorten the reset pulse to ~10 ms.

Ghost / faded image from the previous frame: Always do a full refresh (not partial) on the tri-color (B) variant — it does not support partial refresh. Call `Clear()` before drawing, and refresh at least once every 24 hours.

Red pixels display as black (or vice versa): The two image buffers were swapped. Pass the B/W buffer first, then the red buffer, to `display()`.

SPI device not found on Raspberry Pi: Enable SPI with `raspi-config`. Check that `/boot/config.txt` does not disable `spi0-0`. Verify with ``ls /dev/spi*`` — you should see `/dev/spidev0.0`.

Image looks misaligned or shifted: Check width/height in your code. The panel is 296 × 128 — the library swaps these in landscape mode, so buffers are (height, width). Also verify the image is a 1-bit PIL image.

Display damaged after long power-on with no refresh: The panel must not be parked at high voltage. Always call `sleep()` after your last refresh. If the panel was left powered for days without refresh, some damage may be permanent.

Flickering takes longer than expected: Full refresh of a tri-color panel is ~15 s by design — this is normal. Flickering is the waveform driving particles to their correct positions; shorter refreshes sacrifice image quality.

9. References & Further Reading

- **Waveshare Wiki — 2.9" e-Paper Module (B) Manual**

[https://www.waveshare.com/wiki/2.9inch_e-Paper_Module_\(B\)_Manual](https://www.waveshare.com/wiki/2.9inch_e-Paper_Module_(B)_Manual)

- **Waveshare Product Page — 2.9" e-Paper Module (B)**

<https://www.waveshare.com/2.9inch-e-paper-module-b.htm>

- **GitHub — Waveshare e-Paper driver repo (all MCUs)**

<https://github.com/waveshare/e-Paper>

- **GitHub — soonuse/gdew029z10_2.9inch_e-paper_b (Arduino/RPi/STM32 libraries)**

https://github.com/soonuse/gdew029z10_2.9inch_e-paper_b

- **ESPHome — Waveshare e-Paper integration docs**

https://esphome.io/components/display/waveshare_epaper/

- **Waveshare e-Paper FAQ**

https://www.waveshare.com/wiki/E-Paper_FAQ

Reference compiled from Waveshare's official wiki and community driver libraries. Always consult the current wiki page for the exact revision of your module.